VRS41 — Scan-to-Scan Wavelength Offset

Diagnosis, ranked hypotheses, and a reproduction protocol

Item	Entry
Subject	Rigid wavelength offset between groups of otherwise identical scans
Observed	2026-07-29, ~15:04–15:27, He line region, 587.5 → 588.2 nm
Offset magnitude	approx. 0.07 nm
Configured Slip	0.080 nm
Status	Cause not yet proven. Averaging ruled out. Three code defects found.
Deliverables	This document, `offset_probe.py`
Software changes made	**None.** Diagnosis only, as instructed.

1. What was observed

Sixteen scans of the same line, all driven in the same direction (587.5 → 588.2 nm, step 0.01 nm), returning to the start between scans. They fall into two groups:

- Within a group, the curves are **near-congruent** — same peak position, same width, same shape.
- Between the groups, every curve is **rigidly translated** in wavelength by about **0.07 nm**. No change in width, height, or flank steepness.
- The group boundary falls between 15:12:01 and 15:12:42, which is also when the Avg-fraction setting was changed from 1 % to 50 %.
- A separate static test was then run: **at standstill, nothing moves.** The signal does not creep after a move completes.

Three curves are timestamped 15:10:51, 15:10:55 and 15:10:56 — four and five seconds apart. A full scan of 71 points cannot complete in that time, so these are aborted runs. They should be excluded from any comparison, but their existence matters (see Section 5, Defect A).

2. What this rules out, and why

2.1 The averaging window is not the cause

This is now a logical exclusion, not a numerical one.

> Measurement **duration** can only affect the recorded wavelength if something **moves during the measurement**. The static test shows nothing moves. Therefore no averaging setting — window length, sample count, or fraction — can shift the wavelength axis.

This holds regardless of what Timebase and Pre-settle were actually set to at the time, which is why it supersedes the arithmetic argument from the earlier analysis.

The correlation with the 1 % → 50 % change is real but **not causal**. Something else happened in the same 41-second gap.

2.2 Thermal drift is not the cause

Congruence within a group, held over 8 and 14 minutes respectively, is incompatible with a monotonic drift. A drift would smear the curves within each group, not stack them.

2.3 Random backlash scatter is not the cause

Scatter is random by definition. This is a single step change that then holds.

2.4 Detector or electrical time constants are not the cause

Per-point wall time is approximately 0.9 s, dominated by the move, not the read. A change in the averaging window of a few tens of milliseconds is a fraction of a percent of that. For a lag model to produce 0.07 nm of *difference* it would need a baseline lag of order 2 nm, which is absurd. Ruled out on magnitude.

2.5 Mechanical relaxation after the position report is not the cause

This was the leading hypothesis before the static test. The test killed it: if the grating settled after the drive reported "in position", the signal would creep at standstill. It does not.

3. What remains

Something changed the relationship between **where the grating is** and **what wavelength the software calls it**, once, in that 41-second gap, and it then stayed changed.

The offset magnitude is the strongest clue available. Observed approximately 0.07 nm against a configured Slip of exactly 0.080 nm. That is one unit of backlash compensation, not an arbitrary number.

Independent corroboration from the code itself: the Free Run re-anchoring fix carries this comment about the bug it repaired —

> "*...this is the line that produced the logged* `587.000 -> 586.918 nm` *every sweep.*"

That difference is **0.082 nm**. One Slip. The codebase has already had one one-Slip anchoring offset of exactly this shape, in a different code path.

4. Code audit: three sources of positional truth

The application maintains three representations of where the instrument is, and mixes them within single expressions.

Source	What it is	Where it is used
`read_position()`	Encoder count. The only ground truth.	Only to build `target_abs`
`state['current_nm']`	Bookkeeping. Set to the **commanded** target.	Display, plot labels, CSV
`entry_current.get()`	**GUI text field, rendered to 3 decimals.**	**Origin of every delta calculation**

The pattern, which appears in `goto_worker` and again in `scan_worker`'s jump-to-start:

```
current_steps  = read_position()                          # encoder
current_nm_ui  = float(refs['entry_current'].get() or ...)   # text field
delta_nm       = target_nm - current_nm_ui                  # from the text field
target_steps   = current_steps + steps_for_delta_nm(delta_nm)
```

The encoder is read, but it is not consulted for the delta. The delta comes from a rounded GUI string. Two coordinate systems, added together in one line.

The rounding itself is negligible — one picometre. The structural consequence is not: **any error that ever enters that field is permanent and is inherited by every subsequent move.**

---

5. Three defects

Defect A — the stepped scan does not re-anchor after an abort; Free Run does

Free Run explicitly recovers the true position, however the sweep ended:

```
# Re-anchor to the REAL position after the sweep, however it ended.
final_pos = read_position_or_none()
real_nm   = s_nm + steps_to_delta_nm(final_pos - nm_anchor_steps)
```

`scan_worker` has no equivalent. On an interrupted step it does:

```
if not wait_until_position(...):
    safe_stop(); log("[STOP] Scan interrupted"); break
```

`current_nm` and `entry_current` are left at the last **plotted** point, while the drive continues along its DEC ramp past it. The fix was applied to one of the two paths.

Severity: poisons the origin of every later move by the coast distance. **Relevance to the offset:** direct. The three aborted runs at 15:10:5x are exactly this situation.

Defect B — Go To continues driving after Stop, from a stale origin

```
else:
    log("[STOP] Motion interrupted")
    recover_after_stop()
    current_nm_ui = float(refs['entry_current'].get() or ...)  # never updated
    delta_nm      = target_nm - current_nm_ui                  # nearly the full trip again
    send_cmd(f"LR{rel_steps}"); send_cmd("M")
    ...
    state["current_nm"] = target_nm                            # asserts the target
```

Pressing Stop during a long move causes the app to **re-issue the move**, with the delta computed from the original starting point even though part of the distance has already been covered. The drive overshoots by roughly the distance already travelled, and the software then records the target as the actual position.

This is the reported second bug: after Stop on a long move, the dial and the software disagree. The error scales with the length of the move, which matches "over long distances".

Defect C — the grid/slack split, and the prime suspect for the offset

In `scan_worker`:

```
comp = reversal_compensation_steps(direction)
grid_slack_steps += comp
target_abs = grid_anchor_steps + rel_step_steps * grid_index + grid_slack_steps
```

`comp` enters the **position target**. It does not enter `pos_nm`, the value plotted and exported. That is deliberate, and correct **under the assumption** that the compensation is pure slack take-up and involves no optical travel.

If that assumption is wrong, or if the Slip value is wrong, the entire scan is rigidly translated with respect to its own labels by up to one Slip: **0.080 nm**.

Whether `comp` is applied at all depends on `state['last_move_direction']` — and that variable is mutated by **every** move in the program: Go To, Jog, Reference Run, the calibration dialog, the scan's own jump-to-start. A single jog or correction between two scans flips it, so one scan receives compensation that the previous one did not.

Predicted signature: groups of scans, congruent within a group, rigidly offset between groups by one Slip. That is the observation.

---

6. Ranked hypotheses and their signatures

#	Hypothesis	Predicted signature	How it is falsified
H1	Defect C: compensation enters the target but not the label	Offset is exactly one Slip; disappears when Slip = 0	Test T2
H2	Defect A: an abort poisoned the origin; all later scans inherit it	Offset appears after an abort and **persists**; magnitude equals the coast distance, not the Slip	Test T3
H3	Direction bookkeeping flipped by an intervening jog or Go To	Offset appears after a jog and persists	Test T1
H4	The offset is real optical motion, not a labelling error	Offset present in the **encoder** frame too	Probe, automatically
H5	Something else in the manual workflow	Nothing above reproduces it	All tests negative

H1, H2 and H3 all produce "congruent within a group, rigidly offset between groups". They are distinguished by **what triggers the step** and by **whether the magnitude equals the Slip**.

7. The decisive measurement

A peak position on its own cannot separate a labelling error from real motion, because both move the peak. Two coordinate systems are needed, and one of them must be independent of the software's bookkeeping.

nm_app — the label, computed exactly as `scan_engine` computes it: an absolute grid of `start + step × index`, with compensation folded into the position target but not the label.

nm_enc — derived from the encoder: `anchor_nm + steps_to_delta_nm(POS − anchor_steps)`, where the anchor is established **once at the start of the whole session and never re-established**.

The session-global anchor is the essential design point. It puts every scan of the run into one common frame, so peaks from different scans are directly comparable.

Result	Conclusion
`nm_enc` peak constant, `nm_app` peak jumps	**Software bookkeeping.** The grating is repeatable; the wavelength assignment is not. H1/H2/H3.
`nm_enc` peak jumps as well	**Mechanics or drive.** The grating really goes elsewhere. H4.
Neither jumps	The perturbations tested do not cause it. H5; the trigger is elsewhere in the manual workflow.

This decision does not require any hypothesis to be correct in advance, which is why it is worth the measurement time.

Use the **centroid** of the upper half of the peak, not the maximum. With one DAQ read per point the top of the peak is noisy and the argmax hops between adjacent points — a one-step scatter of its own, which at step 0.01 nm is 10 pm of avoidable noise on a 70 pm effect.

8. Manual test protocol

Run these before, or instead of, the automated probe if time is short. T1 and T2 are the two that matter most.

Preconditions for every test

Check	Done
Lamp warmed up, at least 30 min	☐
Only one program on the COM port — no Motion Manager, no second instance	☐
Reference Run performed after Connect, so `last_move_direction` is defined	☐
Console log open and being captured to a file for the whole session	☐
DAQ source shown as NI-DAQ, not SIMULATED	☐
Timebase at least 200 ms, Pre-settle at least 50 ms	☐
Ambient temperature noted: ______ °C	☐

The Timebase requirement is not cosmetic. At 10 ms Timebase a single DAQ read overruns the whole budget, so the averaging settings have no effect at all and every point is a single unaveraged read.

T1 — Reproduce the offset without touching averaging

Purpose: show that the trigger is a positional event, not an acquisition setting. Averaging is held fixed throughout.

1. Scan. Scan again. → expect congruent.
2. Jog +, then Jog −, returning to roughly the same place.
3. Scan. Scan again. → **if these are offset from step 1 by about one Slip, H3 is confirmed and averaging is definitively out.**
4. Scan a third time. → does the offset persist, or spring back?

Run	Peak λ (nm)	Offset vs run 1 (nm)	Notes
1 baseline		0	
2 baseline			
3 after jog			
4 after jog			
5 after jog			

T2 — Slip = 0

Purpose: falsify H1 directly.

Set the Slip field to `0`. `reversal_compensation_steps()` then returns 0 for every move. Repeat T1 exactly.

- Offset **gone** → the compensation arithmetic is the cause. H1 confirmed.

- Offset **still present** → the compensation is not the cause. H1 dead, and my primary hypothesis was wrong.

Run	Peak λ (nm)	Offset vs run 1 (nm)
1 baseline, Slip = 0		0
2 baseline, Slip = 0		
3 after jog, Slip = 0		
4 after jog, Slip = 0		

T3 — Abort, then scan

Purpose: test H2, and exercise Defect A.

1. Two baseline scans.
2. Start a Go To over a long distance — 5 nm or more — and press Stop partway.
3. Two more scans, without any correction in between.
4. Record whether the offset appears, and whether it persists into a third and fourth scan.

The magnitude here is the discriminator: an abort offset is the **coast distance**, which depends on the DEC ramp and the speed at the moment of Stop. It has no reason to equal the Slip.

T4 — Stop on a long move (Defect B, separate bug)

Purpose: confirm the reported dial-versus-software disagreement, and record its size.

1. Note the mechanical dial reading and the software λ. They should agree.
2. Go To a target at least 20 nm away.
3. Press Stop roughly halfway.
4. Record: dial reading, software λ, and the console log lines.
5. Watch for whether the drive **continues moving after the Stop** — Defect B predicts a second move is issued and the target is reached anyway.

	Dial (nm)	Software λ (nm)	Difference
Before			
After Stop			

T5 — Console log audit

`reversal_compensation_steps()` logs every application:

```
[SLIP] Reversal -1->+1: adding 0.080 nm (+NNN steps) backlash compensation.
```

The log from the 2026-07-29 session therefore already records which scans received compensation and which did not. If that line is present for one group and absent for the other, H1 is confirmed from data already on disk — no rig time required.

---

9. The automated probe — and whether it actually helps

`offset_probe.py` accompanies this document.

Honest answer: yes, with one clearly bounded limitation

Where it helps. It removes the operator from the loop. The effect under investigation is roughly 70 pm and the manual scatter of "jog a bit, scan again, read the peak off the plot" is of the same order. The probe fixes the approach direction, the perturbation, and the peak metric across every run, brackets each perturbation with baseline pairs, and computes the offsets with the same estimator every time. It also records, per point, both wavelength frames plus the raw encoder count, the cumulative compensation, and `last_move_direction` — none of which the GUI's own CSV or log contains today. That per-point label-versus-encoder residual is the actual diagnostic, and it is not obtainable from the GUI at all.

Where it does not help, stated plainly. It does not import or drive `gui_main` or `scan_engine`. Those are welded to Qt widgets — `refs['entry_current']`, `refs['btn_pause'].configure(...)` — and stubbing widgets to run them headless would put the stubs into the measurement path, so a clean result would prove nothing about the real application. The probe therefore re-implements the scan loop transparently from the same low-level primitives the app uses.

The consequence: **the probe can prove or disprove a physical offset, and can show whether the grid/slack arithmetic accounts for it. It cannot prove what `gui_main` does, because it does not run `gui_main`.** Defects A and B live in GUI-thread code paths that only the GUI exercises. Those need T3, T4 and the console log.

So: probe for the physics and the arithmetic, manual tests plus log for the application. They cover different halves and neither replaces the other.

What it does

Twelve scans in a fixed matrix, with baseline pairs bracketing each perturbation:

```
01 baseline          07 jog
02 baseline          08 after-jog
03 avg-change        09 after-jog-2
04 after-avg         10 abort
05 avg-restore       11 after-abort
06 after-restore     12 after-abort-2
```

The averaging perturbation is included deliberately, even though Section 2.1 rules it out on principle — a prediction that is not tested is not a prediction.

Usage

```
# 1. Verify the analysis maths. No hardware, no application modules needed.
python offset_probe.py --selftest

# 2. Dry run against the virtual drive, to check it survives contact with
#    the real module layout. Results characterise the simulator only.
python offset_probe.py SIM --center 587.80 --span 0.40 --step 0.02

# 3. The real run. Set --center to the peak you actually see, not the
#    textbook line.
python offset_probe.py COM3 --center 587.80 --span 0.40 --step 0.01 \
    --timebase-ms 300 --presettle-ms 50 --slip-nm 0.080 \
    --outdir probe_2026-07-30

# 4. Test T2, automated: same matrix with compensation forced off.
python offset_probe.py COM3 --center 587.80 --no-compensation \
    --outdir probe_2026-07-30_noslip
```

Run it from the directory containing `motion.py`, `protocol_faulhaber.py`, `daq.py` and `device_constants.py`. It resolves the primitives it needs by name across those modules and prints where it found each one, so a module-layout skew produces a clear message rather than a wrong answer.

Estimated duration for step 0.02 nm and span 0.40 nm: 21 points per scan, 12 scans, plus repositioning — roughly 10 to 15 minutes at 0.9 s per point.

Output

File	Contents
`scan_NN_<tag>.csv`	One row per point: `nm_app`, `nm_enc`, `label_minus_encoder_nm`, `pos_steps`, `volts`, `meas_dur_s`, `reads`, `grid_slack_steps`, `last_move_direction`
`summary.csv`	One row per scan: peak in both frames, offsets versus run 1, label-versus-encoder residual, peak height, baseline
`probe.log`	Full transcript, including every `[SLIP]` line and every move that failed to reach its target

It ends by printing a verdict from the peak spread in each frame, per the table in Section 7.

What it deliberately does not do

- No `SAVE` or `EEPSAV`. Nothing on the drive is made permanent.
- No modification of any application source or settings file.
- Backlash compensation is taken from an explicit `--slip-nm` argument rather than from the GUI field, so it cannot silently be zero and thereby suppress the very effect under test. What `reversal_compensation_steps()` *would* have returned is recorded alongside, and a disagreement is logged as a warning.

Verification status

The analysis half is verified against synthetic data: an 80 pm offset injected into the label frame only is recovered as 80.2 pm in `nm_app` and 0.2 pm in `nm_enc`, and the verdict logic fires correctly. Run `--selftest` to reproduce.

The hardware half is **not** verified — this was written without access to the current `.py` files, against the module and function names visible in the project history. Run step 2 above before step 3 and expect to fix an import name or two.

---

10. What to send back

In priority order.

1. **The console log of the 2026-07-29 session, 15:00 to 15:30, complete.** Specifically every `[SLIP]`, `[STOP]`, `[DONE]`, `[CAL]` and `[FREE RUN]` line. This may settle H1 with no rig time at all (test T5).
2. **CSV headers** of one curve from each group — the Timebase and Pre-settle actually used, rather than what the panel shows now.
3. **T1 and T2 results.** Between them they either confirm or kill the primary hypothesis.
4. **The probe output directory**, if it runs.
5. **T4 result** for Defect B, which is an independent bug regardless of how the offset question resolves.

Defects A and B are real and should be fixed whatever the outcome. I would rather see item 1 before touching code, because if H1 is confirmed the fix is in a different place than if H2 is.

Appendix — code sites referenced

Defect	Module	Location
A	`scan_engine.py`	`scan_worker`, the `if not wait_until_position(...)` break; compare with the re-anchor block in `free_run_worker`
B	`scan_engine.py`	`goto_worker`, the `else:` branch after `[STOP] Motion interrupted`
C	`scan_engine.py`	`scan_worker`, `grid_slack_steps` accumulation versus `pos_nm` advance
C	`motion.py`	`reversal_compensation_steps()`, and the `state['last_move_direction']` side effect
Mixed truth	`scan_engine.py`	`goto_worker` and `scan_worker` jump-to-start: `entry_current.get()` as the delta origin